

HANDS-ON 6 [Intermediate]

FastAPI — Path Parameters, Pydantic & Async Endpoints

Topics Covered:

FastAPI Project Setup✓

Path & Query Parameters✓

Pydantic Models for Validation✓

Async / Await in FastAPI✓

Automatic OpenAPI / Swagger Docs✓

Response Models✓

FastAPI is a modern, high-performance framework built on ASGI that auto-generates OpenAPI documentation and uses Pydantic for data validation. You will rebuild the Course API in FastAPI and experience its developer-friendly features.

SETUP

pip install fastapi uvicorn sqlalchemy then uvicorn main:app --reload to run

Task 1: FastAPI Setup and Pydantic Schemas

Goal: Scaffold a FastAPI application and define Pydantic schemas for request/response validation.

Steps:

57.

Create main.py with: from fastapi import FastAPI; app = FastAPI(title='Course Management API', version='1.0'). Add a root route @app.get('/') that returns {'message': 'API running'}.

58.

Create `schemas.py`. Define Pydantic BaseModel classes: `CourseCreate` (`name`, `code`, `credits`, `department_id`), `CourseUpdate` (all fields optional using `Optional[]`), `CourseResponse` (all fields plus `id` — this is the response schema).

59.

Define a `DepartmentResponse` schema that nests a list of `CourseResponse` objects to demonstrate Pydantic nested models.

60.

In `main.py`, add a `POST /api/courses/` endpoint with signature: `async def create_course(course: CourseCreate)`. FastAPI automatically validates the request body against the Pydantic model and returns 422 if validation fails.

61.

Visit <http://127.0.0.1:8000/docs> — observe the auto-generated Swagger UI showing your endpoints with request/response schemas.

Hint:

-

Pydantic validates data at parse time — if the request body does not match the schema, FastAPI returns a detailed 422 Unprocessable Entity response automatically.

-

`Optional[str] = None` marks a field as optional with a default of `None`. Use this in update schemas where not all fields are required.

Expected Outcome: `/docs` shows the `POST /api/courses/` endpoint with the `CourseCreate` schema. Sending invalid data returns a 422 with field-level error details.

Task 2: Path/Query Parameters and Async Database Access

Goal: Add path and query parameters, and wire up async SQLAlchemy for database access.

Steps:

62.

Add a GET `/api/courses/{course_id}` endpoint. The path parameter `course_id` is automatically validated as an integer by FastAPI.

63.

Add a GET `/api/courses/` endpoint with optional query parameters: `skip: int = 0`, `limit: int = 10`, `department_id: Optional[int] = None`. Use these to implement pagination and filtering.

64.

Set up SQLAlchemy with an async engine using `create_async_engine`. Define a `get_db()` dependency function that yields a database session.

65.

Inject the database session into endpoints using FastAPI's Dependency Injection: `async def get_courses(db: AsyncSession = Depends(get_db))`.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

66.

Implement the full CRUD for courses using async ORM queries: `await db.execute(select(Course))`, `await db.commit()`, etc.

67.

Test pagination: GET /api/courses/?skip=0&limit=2 should return the first 2 courses; GET /api/courses/?skip=2&limit=2 should return the next 2.

Hint:

-

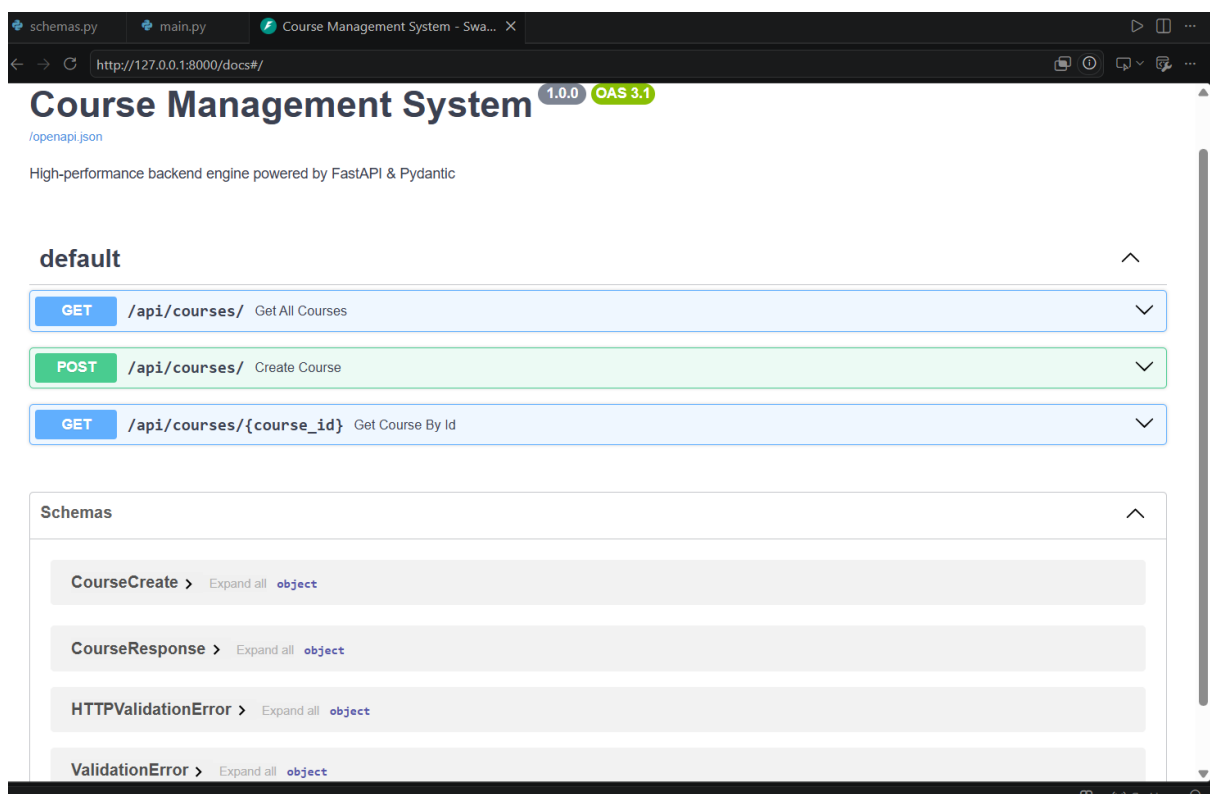
FastAPI's Depends() system is its dependency injection mechanism — get_db() is called automatically before each request and cleaned up after.

-

async def endpoints are handled by uvicorn's async event loop — they do not block other requests while waiting for I/O.

Expected Outcome: GET /api/courses/?limit=2 returns 2 courses. Filtering by department_id returns only courses in that department. All operations are async.

OUTCOME:



schemas.py

main.py

Course Management System - Swa...

http://127.0.0.1:8000/docs#/default/get_all_courses_api_courses_get

curl -X 'GET' \

'http://127.0.0.1:8000/api/courses/' \

-H 'accept: application/json'

Request URL

http://127.0.0.1:8000/api/courses/

Server response

Code

Details

200

Response body

```
{
  {
    "name": "Python Web Frameworks",
    "code": "CS301",
    "credits": 4,
    "id": 1,
    "department_id": 1
  },
  {
    "name": "Relational Databases",
    "code": "CS302",
    "credits": 3,
    "id": 2,
    "department_id": 1
  },
  {
    "name": "Data Structures",
    "code": "19AI590",
    "credits": 3,
    "id": 3,
    "department_id": 2
  }
}
```

Download

Response headers

```
content-length: 251
content-type: application/json
date: Wed, 01 Jul 2026 08:25:33 GMT
server: uvicorn
```

Go Live